

Core

Last updated by | Mennell, Jacob | 19 May 2026 at 11:57 BST

Claims Augmented Triage

Purpose

This wiki introduces the Claims Augmented Triage solution for an external technical audience. It explains what the platform does, how the main components fit together, and where AI supports the wider claims process.

What The Solution Does

Claims Augmented Triage is a document- and workflow-driven platform for handling claims information. It combines core application services, retrieval over stored claim evidence, and AI-assisted extraction and checking.

At a high level, the solution:

- ingests claim-related documents and converts them into retrieval-ready chunks
- stores structured metadata and vector embeddings for downstream search and reasoning
- runs orchestrated extraction and checking workflows over claim evidence
- exposes claim, document, orchestration, and computation capabilities through an API
- provides a web-based user interface for consuming outputs and navigating model-driven claim views

Primary Capabilities

- Document ingestion and chunk enrichment using Azure Document Intelligence and fallback parsing paths
- Retrieval over claim evidence using lexical, semantic, and hybrid retrieval approaches
- AI-assisted extraction workflows with deterministic, agentic, and progressive execution modes
- AI-assisted checks with bounded retry and fail-soft behavior
- REST API access for claims, documents, orchestration, models, feedback, and computation endpoints
- Browser-based user experience for claim exploration, extracted information, checks, and administrative functions

Major Components

- Frontend: React and Vite application served behind NGINX
- API: FastAPI service exposing authenticated REST endpoints under `/api/`
- Main worker: orchestration and AI workflow execution service
- Ingestion worker: document parsing, chunking, embedding, and persistence service

- PostgreSQL with pgvector: operational store plus vector retrieval store
- RabbitMQ: asynchronous messaging backbone for background task execution

Recommended Reading Order

If you are new to the solution, start here:

1. [Solution Overview](#)
2. [System Topology](#)
3. [End-to-End Runflow](#)

Then drill into the relevant domain:

- ingestion: [Document Ingestion](#)
- retrieval: [Chunking, Embedding and Retrieval](#)
- orchestration: [AI Orchestration, Extraction, Checks](#)
- API: [API](#)
- frontend: [Frontend UX](#)
- deployment and operations: [Deployment, Environment and Config](#)

Scope Of This Wiki

This wiki focuses on externally shareable architecture and solution behavior. It does not reproduce internal prompt assets, low-level tuning decisions, or engineering-only implementation detail.

Assumptions / Open Questions

- The repository shows the technical solution structure clearly, but the exact business operating model and external user personas may need refinement with product stakeholders.